

Hacking The Interwebs

Hacking The Interwebs - pdp, gnu citizen.org.

- Published: date not stated
- Original: <<https://www.gnucitizen.org/blog/hacking-the-interwebs/>>
- Preserved from: <https://www.gnucitizen.org/blog/hacking-the-interwebs/> (browser) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Hacking The Interwebs

Sat, 12 Jan 2008 12:57:22 GMT

by [pdp](#)

"With great power comes great responsibility", but those with great power usually aren't that responsible. Nevertheless, we try to be responsible as much as we can. In the following post, ap and I are going to expose some secrets, which may make you question our values at first, will definitely make you feel worried about "Why is all this possible?", and may even make you hate us in your guts for what we have done.

It is important to understand the magnitude of the problem we are planning to talk about, and that we cannot go to any vendor to ask for a solution, because it is not a bug what we have to deal with, but rather a combination of design problems. It is an issue, which needs to be resolved right now and the only way to do that is to go public with whatever we've got on our table.

During the last week we've tried to prepare you for this very moment by exposing [bits and pieces](#) on how UPnP works and why it is so important to keep it in mind when testing and securing networks. We've also [talked](#) about how the Universal Plug and Play can be combined with simple XSS attacks in order to create a powerful mechanism for remotely reconfiguring vulnerable routers without any means of authentication or authorization with the targeted device. Today, we are going to show you that UPnP can be exploited across the Web without the need of XSS at all. This is the next logical, evolutionary step of UPnP exploitation which by far has the highest level of severity.

We've [talked](#) earlier that the UPnP stack consists of several technologies: SSDP (Simple Service Discovery Protocol), GENA (Generic Event Notification Architecture), SOAP (Simple Object Access Protocol) and XML. The UPnP control process starts with the discovery stage. Here, a multicast

SSDP packet is submitted to 239.255.255.250:1900. Any device that listens on this multicast port will then respond with information about their service description if they are happy with the body of the discovery packet. The UPnP control actuator will then read the description and look for available methods. Each method is associated with a control point (URL and a header) and method parameters which may or may not be required. Once the method information is obtained, the UPnP actuator will pick the method that suits best the given task that needs to be performed and submit a SOAP message to the control point in order to actualize it. "This is how UPnP works in general!"

When attacking UPnP from within the network where the UPnP enabled device is located, we pretty much proceed with the method described above. If we want to attack a UPnP enabled device across the Web, then we have a few problems that needs to be solved. First of all, from the Web, we cannot send and process SSDP. SSDP is based on UDP and it deals with multicast packets which is something browsers and Web technologies in general will probably never learn how to work with. The only stage that we can safely perform from the Web is the actual SOAP request, which is the very last stage of the control mechanism described in the previous paragraph.

Adrian did an amazing job [explaining](#) how someone can reconfigure your BT Home Hub router via a pre-auth XSS. In his post, Adrian describes a mechanism where the victim visits a malicious page, which makes use of a XSS vulnerability that exists within the BT Home Hub router, in order to add a portforwarding rule within the targeted device firewall. Once the XSSed SOAP request is actualized, the attacker will be able to get access to an internal service over the portforward. Given the fact that the attacker can change the primary DNS server of the target router, as well, the problem seems to be more then scary and very, very concerning. At this stage you are probably thinking that closing the XSS hole on the router pre-auth pages will definitely solve the problem for good, but I am afraid to inform you that you will be wrong.

To the point: SOAP Messages are nothing but POST requests with `contentType` equal to `application/xml`, a `SOAPAction` header and a request body that complies with the SOAP message format. These three request values cannot be changed with JavaScript unless we deal with the `XMLHttpRequest` object. Though, in order to successfully use this object, we need to comply with the Same Origin Policies (SOP) and that will mean that we need an XSS vulnerability, as Adrian [proposed](#) in his article. However, it is less known that these values can be easily set with Flash. The following code demonstrates the attack vector:

```
[/files/2008/01/test.mxml][/files/2008/01/test.mxml)
```

The Test.mxml Flash Application performs several operations.

- At first, the MXML script creates an `URLRequest` object to the targeted UPnP control point URL. In our case, this is `http://192.168.1.254/upnp/control/igd/wanpppcInternet`, which is the PPP control point of BT Home Hub. Keep in mind that other devices can be exploited as well by changing that URL to match their setup.
- Then we define the request **method** which has to be `POST`.
- The next expression defines the request data. This is the actual SOAP Message which will add the portforwarding rule.

- We need to set the `contentType` to `application/xml`.
- Then we push the `SOAPAction` header into the Array of headers.
- And finally we open the `URLRequest` with `navigateToURL`. The respond will render within `_self`.

Shockwave Flash 9.0 r115 (the latest at the time of writing but not automatically deployed) seems to incorrectly supply the request headers. This may make the attack to fail if you use Firefox, Opera or Safari and the attacked router or UPnP device is picky about CR and CRLF line endings. Earlier flash versions does not have this problem/bug. Keep in mind that most devices will accept the request although the line endings are mixed up a bit.

When the victim visits the malicious SWF file, the above 6 steps will silently execute in the background. At that moment the attacker will have control over the service the portforwarding rule was assigned for. Keep in mind that no XSS is required, it is a matter of visiting the wrong resource at the wrong time. *Also, keep in mind that 99% of home routers are vulnerable to this attack as all of them support UPnP to one degree or another.*

I repeat myself far too much, but I guess I have another opportunity to mention that adding a portforwarding is only one of the many things someone can do to your router. The most malicious of all malicious things is to change the primary DNS server. That will effectively turn the router and the network it controls into a zombie which the attacker can take advantage of whenever they feel like it. It is also possible to reset the admin credentials and create the sort of onion routing network all the bad guys want. *We hope that by exposing this information, we will drastically improve the situation for the future. I think that this is a lot better than keeping it for ourselves or risking it all by given the criminals the opportunity to have in possession a secret which no one else is aware of.*

The only way to protect yourself is to turn off UPnP. Yes, that will make your life harder and probably your skype or msn wont work as flawlessly as before but it is a trade-off you have to learn to live with.